Practical - 4

Problem Definition

• Build a Node/Express backend for a Task Management system.

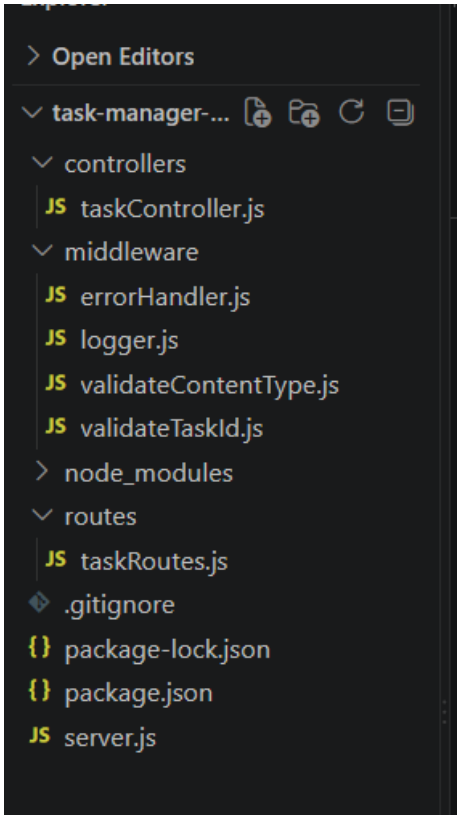
• Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).

• Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.

• Apply a global error handling middleware as the last middleware in the pipeline.

• Use correct HTTP status codes (200, 201, 404, 500) for each response.

Folder Structure:



controllers/taskController.js

```
let tasks = [
  { id: 1, title: 'Learn Express', completed: false },
```

```
  { id: 2, title: 'Build REST API', completed: false }
];
let nextId = 3;

const getAllTasks = (req, res) => {
  res.status(200).json({ success: true, count: tasks.length, data: tasks });
};

const getTaskById = (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) {
    return res.status(404).json({ success: false, error: 'Task not found' });
  }
  res.status(200).json({ success: true, data: task });
};

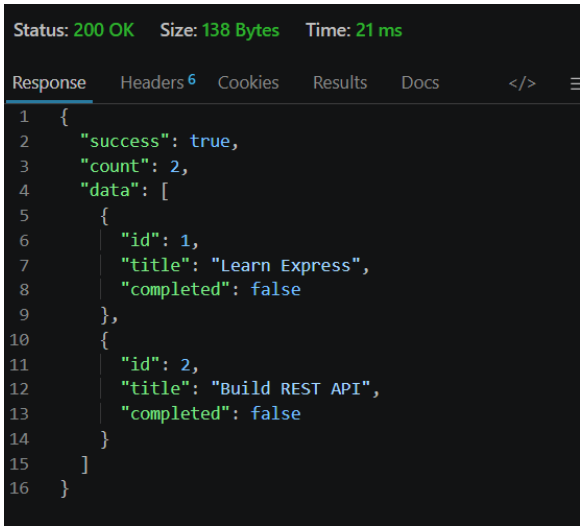
const createTask = (req, res) => {
  const { title, completed } = req.body;
  if (!title) {
    return res.status(400).json({ success: false, error: 'Title is required' });
  }
  const newTask = { id: nextId++, title, completed: completed || false };
  tasks.push(newTask);
  res.status(201).json({ success: true, data: newTask });
};
```

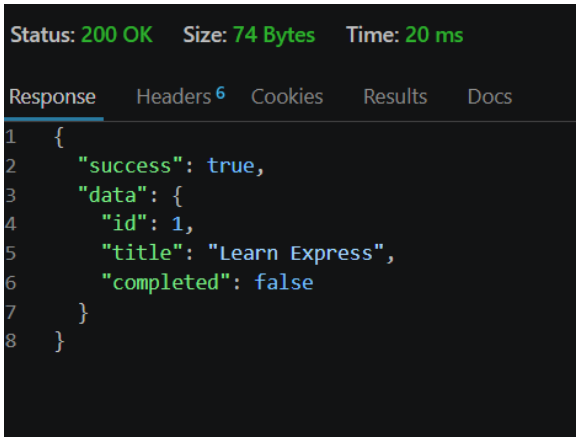
```
const updateTask = (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) {
    return res.status(404).json({ success: false, error: 'Task not found' });
  }
  const { title, completed } = req.body;
  if (title !== undefined) task.title = title;
  if (completed !== undefined) task.completed = completed;
  res.status(200).json({ success: true, data: task });
};

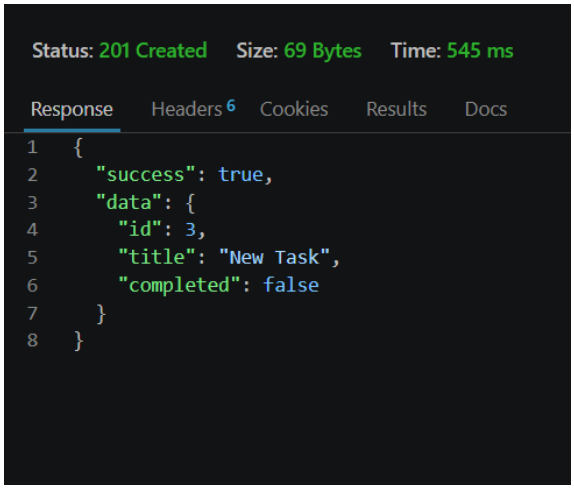
const deleteTask = (req, res) => {
  const index = tasks.findIndex(t => t.id === parseInt(req.params.id));
  if (index === -1) {
    return res.status(404).json({ success: false, error: 'Task not found' });
  }
  tasks.splice(index, 1);
  res.status(200).json({ success: true, message: 'Task deleted successfully' });
};

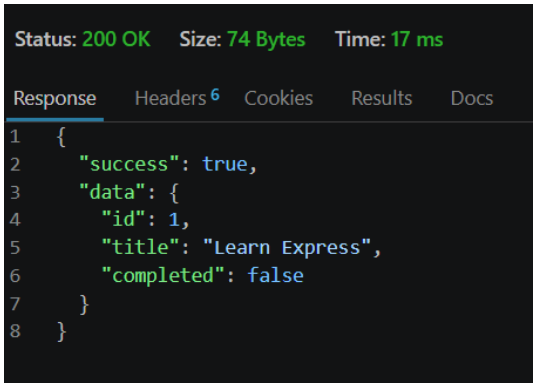
module.exports = { getAllTasks, getTaskById, createTask, updateTask, deleteTask
};
```

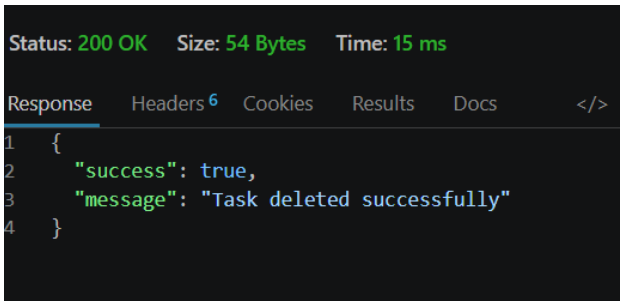
Output:


Status: 200 OK Size: 138 Bytes Time: 21 ms
Response Headers 6 Cookies Results Docs
{
"success": true,
"count": 2,
"data": [
{
"id": 1,
"title": "Learn Express",
"completed": false
},
{
"id": 2,
"title": "Build REST API",
"completed": false
}
]
}


Status: 200 OK Size: 74 Bytes Time: 20 ms
Response Headers 6 Cookies Results Docs
{
"success": true,
"data": {
"id": 1,
"title": "Learn Express",
"completed": false
}
}


Status: 201 Created Size: 69 Bytes Time: 545 ms
Response Headers 6 Cookies Results Docs
{
"success": true,
"data": {
"id": 3,
"title": "New Task",
"completed": false
}
}





middleware/logger.js

```
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next();
};

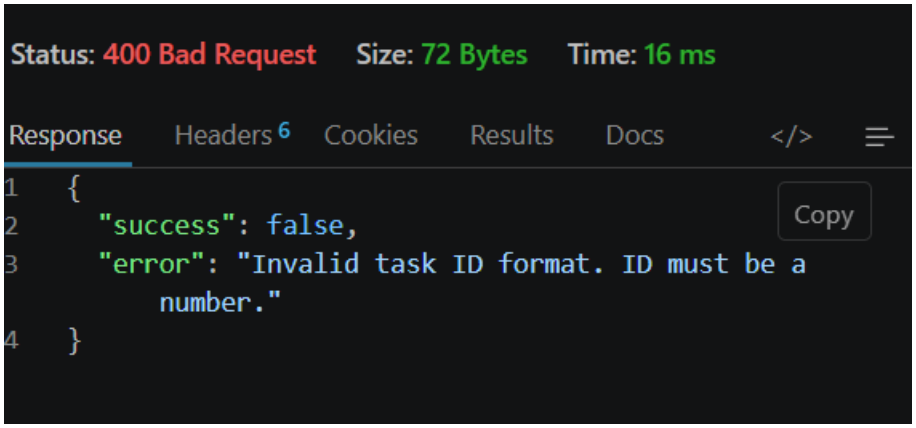
module.exports = logger;
```

middleware/validateTaskId.js

```
const validateTaskId = (req, res, next) => {
  const { id } = req.params;
  if (!/^\d+$/.test(id)) {
    return res.status(400).json({
      success: false,
```

```
    error: 'Invalid task ID format. ID must be a number.'
   });
  }
  next();
};
module.exports = validateTaskId;
```

Output:



middleware/errorHandler.js

```
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    error: 'Something went wrong'
  });
};

module.exports = errorHandler;
```

routes/taskRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getAllTasks,
  getTaskById,
  createTask,
  updateTask,
  deleteTask
} = require('../controllers/taskController');
const validateContentType = require('../middleware/validateContentType');
const validateTaskId = require('../middleware/validateTaskId');

router.get('/', getAllTasks);
router.get('/:id', validateTaskId, getTaskById);
router.post('/', validateContentType, createTask);
router.put('/:id', validateContentType, validateTaskId, updateTask);
router.delete('/:id', validateTaskId, deleteTask);

module.exports = router;
```

server.js

```
const express = require('express');
const taskRoutes = require('./routes/taskRoutes');
const logger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');
```

```
const app = express();
const PORT = 5000;

app.use(express.json());
app.use(logger);

app.use('/tasks', taskRoutes);

app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    path: req.originalUrl
  });
});
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Output:

```
PS C:\Users\Bhumi> cd "E:\CHARUSAT\SEM - 5\AWDF\task-manager-api"
PS E:\CHARUSAT\SEM - 5\AWDF\task-manager-api> node server.js
Server running on port 5000
```

Supplementary Problem :

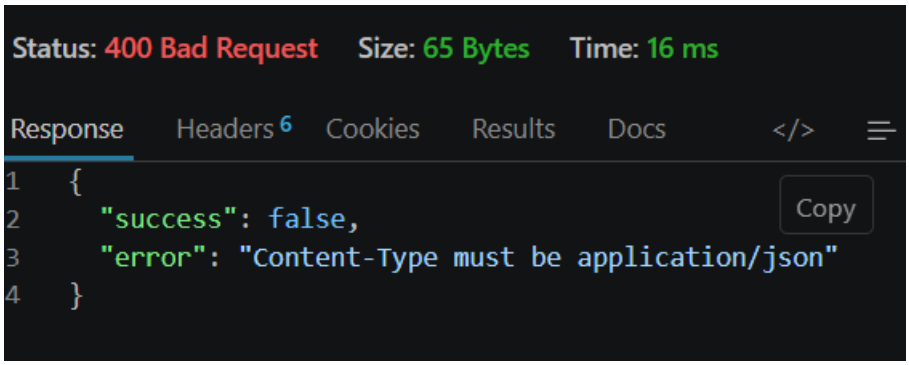
Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.

middleware/validateContentType.js

```
const validateContentType = (req, res, next) => {
  if (req.method === 'POST' || req.method === 'PUT') {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        success: false,
        error: 'Content-Type must be application/json'
      });
    }
  }
  next();
};

module.exports = validateContentType;
```

Output:



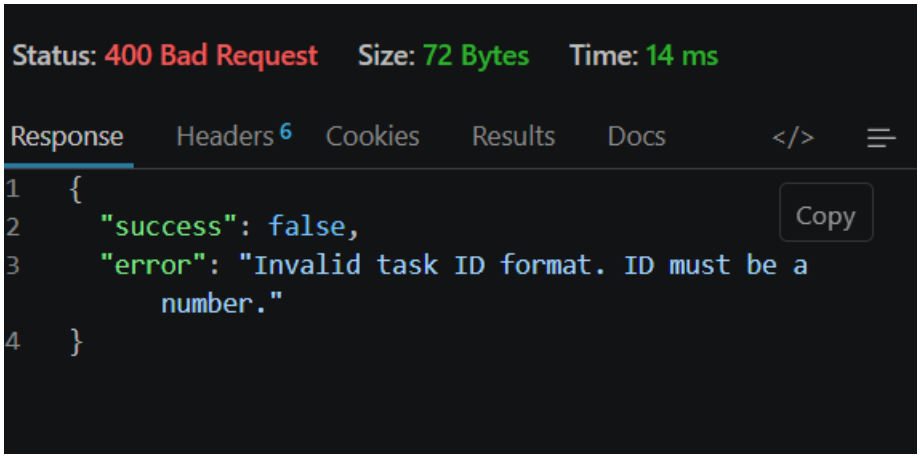
2. Add a route-specific middleware that validates the task ID format before reaching the controller.

middleware/validateTaskId.js

```
const validateTaskId = (req, res, next) => {
  const { id } = req.params;
  if (!/^\d+$/.test(id)) {
    return res.status(400).json({
      success: false,
      error: 'Invalid task ID format. ID must be a number.'
    });
  }
  next();
};

module.exports = validateTaskId;
```

Output:



3. Add a 404 handler for undefined routes that returns a structured JSON response.

server.js

```
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    path: req.originalUrl
  });
});
```

Output:



Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF